Advanced Empirical Research Methods

MT25 Assignment 1 - Part 1

Candidate Number: 1091806

2026-03-25

Contents

Part 1 **2**

Exercise 1: **2**

Question 1 2

Question 2 3

Question 3 3

Question 4 4

Exercise 2: **6**

Question 1 6

Question 2a 6

Question 2b 6

Question 2c 6

Appendix **7**

Part 1 Exercise 1, Julia code 7

Part 1 Exercise 2, Julia code 14

- Exercise 2 Main 14

- Exercise 2 Parameters 17

- Exercise 2 Value Function Iteration 18

- Exercise 2 Kolmogorov-Forward Equation 21

Part 1

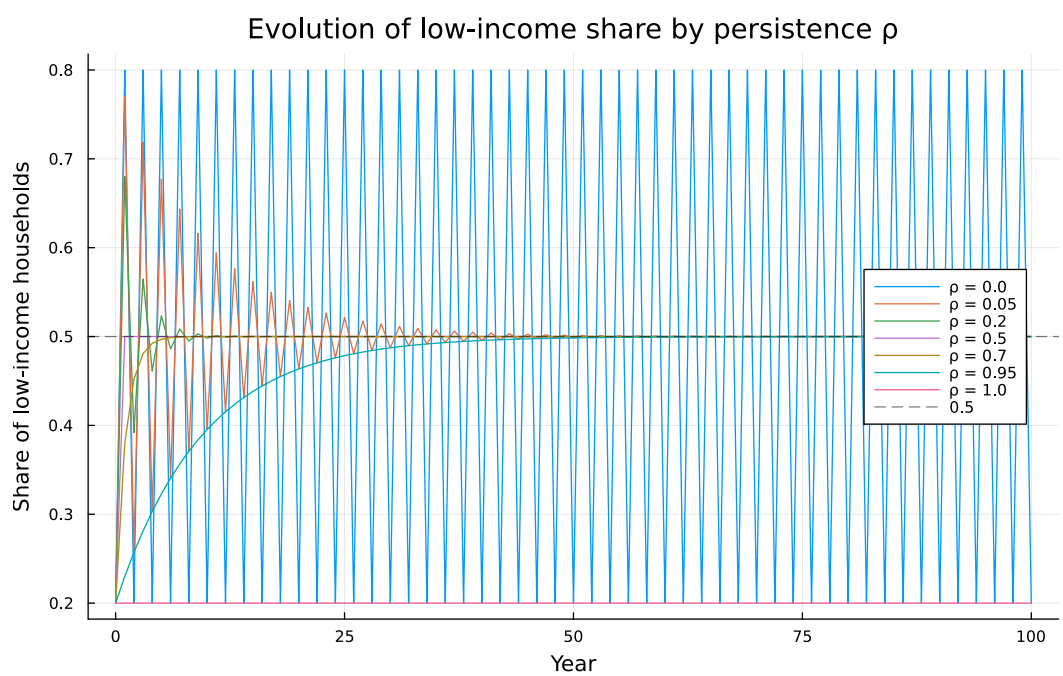
Exercise 1:

Question 1

Income follows a two-state Markov chain with symmetric persistence $\rho_L = \rho_H = \rho$ and transition matrix:

$$P = \begin{pmatrix} \rho & 1-\rho \\ 1-\rho & \rho \end{pmatrix}$$

Starting from $P(y = y_L) = 0.2$, the figure below shows the share of low-income households over a 100-year horizon for each value of ρ.



Explanation:

For all $\rho \in (0, 1)$, the share of low-income households converges to 0.5, which is the unique stationary distribution of this symmetric Markov chain, but higher persistence slows the rate of convergence.

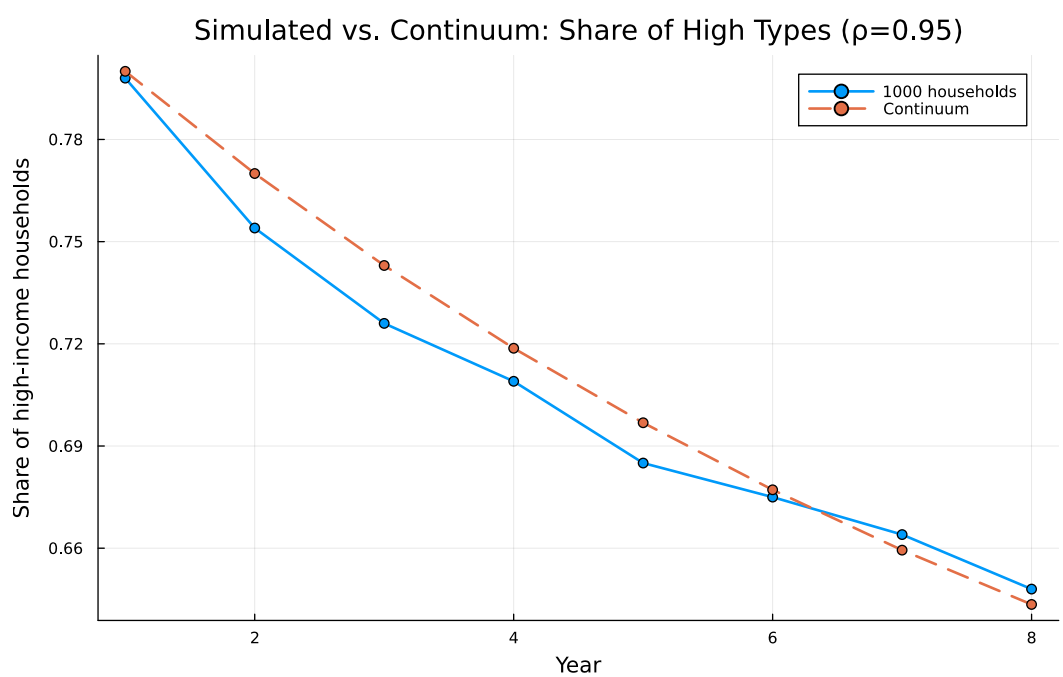
The two edge cases behave differently: $\rho = 0$ causes full state-switching each period (so the share oscillates before settling), while $\rho = 1$ makes both states absorbing, so the initial distribution $P(y_L) = 0.2$ is permanent.

Question 2

ρ	$\mathrm{Var}(\Delta \log y)$
0.00	0.48045
0.05	0.45643
0.20	0.38436
0.50	0.24023
0.70	0.14414
0.95	0.02402
1.00	0.00000

Labour income volatility, measured as $\mathrm{Var}(\Delta \log y)$, is maximised at $\rho = 0$ since households switch between income states every period, making changes in income most frequent. However, $\rho = 0$ does not generate the most risk: when income alternates deterministically, households can anticipate future income and smooth consumption via bond savings, whereas for intermediate or high values of ρ, income shocks are harder to insure against because the current state provides less (or too much persistent) information about future income.

Question 3



The simulated share of high-income types closely tracks the continuum prediction but does NOT match it exactly, because a finite sample of 1000 households is subject to sampling variation whereas the continuum result holds by the law of large numbers over a unit mass of agents.

Question 4

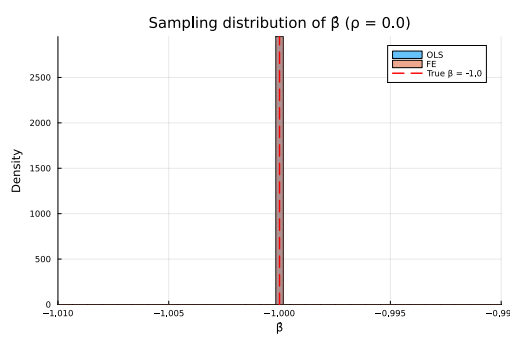
Using the simulated panel data from Q3, I regress current income on lagged income via pooled OLS and fixed effects (implemented via within-transformation demeaning), using only linear algebra as instructed.

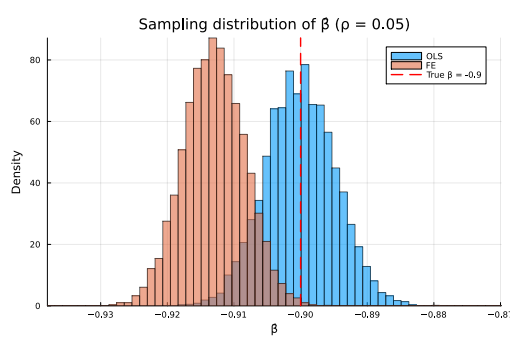
$$y_{i,t} = \alpha_{OLS} + \beta_{OLS} y_{i,t-1} + u_{i,t}$$

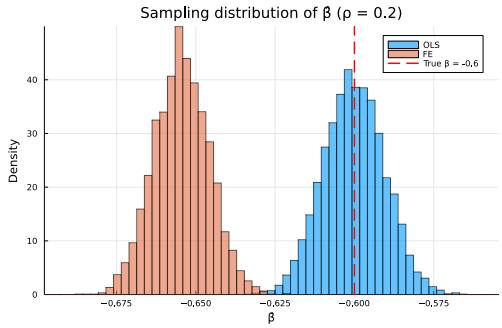
$$\tilde{y}_{i,t} = \beta_{FE} \tilde{y}_{i,t-1} + \tilde{u}_{i,t}, \quad \tilde{x}_{i,t} = x_{i,t} - \bar{x}_i$$

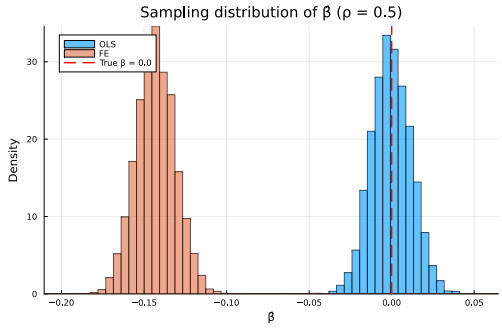
Coefficient	Estimate
α_{OLS}	0.1535
β_{FE}	0.5690

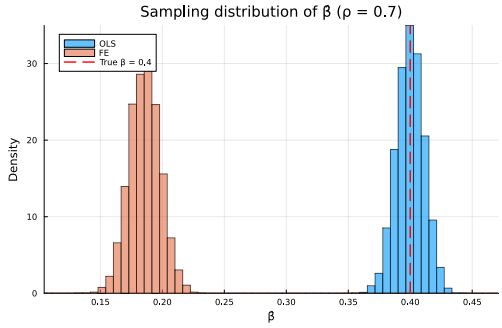
The true population regression slope is $\beta = 2\rho - 1 = 0.9$, since $E[y_{t+1}|y_t]$ is linear in y_t with slope $2\rho - 1$ when $y \in \{1, 2\}$. OLS is close to this true value because the DGP contains no individual heterogeneity, so $E[y_{i,t-1} u_{i,t}] = 0$ and OLS is consistent. The FE estimator suffers from Nickell bias: demeaning over a short panel ($T = 8$) induces a mechanical negative correlation between $\tilde{y}_{i,t-1}$ and $\tilde{u}_{i,t}$, biasing $\hat{\beta}_{FE}$ substantially downward. As the Monte Carlo simulations below confirm, this downward bias increases systematically with ρ.

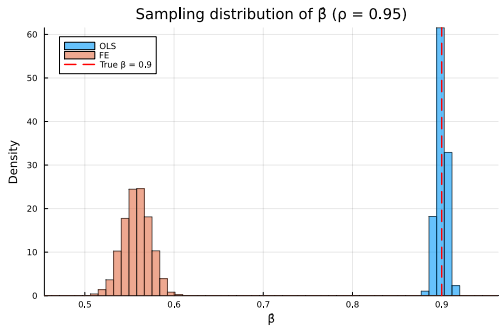





Sampling distribution of β (ρ = 0.2)
OLS
FE
True β = -0.6
Density
β


Sampling distribution of β (ρ = 0.5)
OLS
FE
True β = 0.0
Density
β


Sampling distribution of β (ρ = 0.7)
OLS
FE
True β = 0.4
Density
β


Sampling distribution of β (ρ = 0.95)
OLS
FE
True β = 0.9
Density
β

Exercise 2:

Question 1

ρ	Asset Supply
0.00	-0.275510
0.05	-0.258511
0.20	-0.185725
0.50	0.027987
0.70	0.132477
0.95	-0.443872
1.00	-0.500000

For $\rho = 1$ the transition matrix is the identity, making both income states absorbing. There is no unique stationary distribution, so the iterative procedure fails to converge. However, since $\beta(1+r) = 0.85 \times 1.1 = 0.935 < 1$, households are impatient and face a declining consumption path. Without income risk, there is no precautionary savings motive, so both types eventually borrow to the constraint, giving $B = -0.5$.

Question 2a

Table of counterfactual aggregate asset supply:

	$B(\rho = 0, c = c_{.5})$	$B(\rho = .95, c = c_{.5})$
Asset Supply	-0.11499999999999998	0.5655346767345566

Question 2b

Table of decompositions:

ρ	T_{total}	T_1 (%)	T_2 (%)
0.00	-0.3035	52.9	47.1
0.95	-0.4719	213.9	-113.9

Question 2c

Although the stationary *marginal* distribution over income is always 50/50 regardless of ρ, changing ρ alters the *joint* dynamics of how households move between income states, which affects how long they spend in consecutive high- or low-income spells and thus where they sit on the asset grid. With higher persistence, high-income households accumulate more assets over longer spells before switching, while low-income households remain borrowing-constrained for longer — shifting the stationary distribution over (b, y) and hence aggregate asset supply, even with the same consumption policy.

Appendix

Part 1 Exercise 1, Julia code

```
# Exam MT 2025 Assignment 1
# Part 1 Exercise 1

# Load packages
using Pkg
using Plots
using Random
using Statistics
using LinearAlgebra

# ==============================================================================
# Question 1
# ==============================================================================

# === Set-up ===
# Income states

    yL = 1.0

    yH = 2.0

# Values of persistence to consider

    ρ_vals = [0.0, 0.05, 0.2, 0.5, 0.7, 0.95, 1.0]

# Time horizon
    T = 100

# Initial distribution: P(y = yL) = 0.2
    π0 = [0.2, 0.8]  # [P(yL), P(yH)]using Pkg

# Transition matrix for symmetric persistence ρ:
#       yL      yH
# yL [  ρ     1-ρ  ]
# yH [ 1-ρ     ρ   ]
    function transition_matrix(ρ)
        return [ρ (1-ρ); (1-ρ) ρ]
    end

# Evolve distribution forward: π(t+1) = πt' * P
# (row vector times transition matrix)
    function evolve_distribution(π0, P, T)
        shares_low = zeros(T + 1)
```

```
        shares_low[1] = π0[1]
        πt = copy(π0)
        for t in 1:T
            πt = πt' * P |> vec
            shares_low[t + 1] = πt[1]
        end
        return shares_low
    end

# Compute and plot
    Pkg.add("Plots")
    plt = plot(
        xlabel = "Year",
        ylabel = "Share of low-income households",
        title = "Evolution of low-income share by persistence ρ",
        legend = :right,
        size = (800, 500),
        linewidth = 2
    )

    for ρ in ρ_vals
        P = transition_matrix(ρ)
        shares = evolve_distribution(π0, P, T)
        plot!(plt, 0:T, shares, label = "ρ = $ρ")
    end

# Add reference line at 0.5

    hline!(plt, [0.5], color = :black, linestyle = :dash, label = "0.5", alpha = 0.5)

# ==============================================================================
# Question 2
# ==============================================================================

# At the stationary distribution (50/50), Pr(switch) = 1-ρ for each type
# Δlog(y)   {0, log(2), -log(2)}

    Δ = log(2)

    println("ρ \t\t Var(Δlog y)")

    println("-"^30)

    for ρ in ρ_vals
        # Stationary shares
            p_L = (ρ == 1.0) ? 0.5 : 0.5  # symmetric chain
```

```
            p_H = 1 - p_L

        # Joint probabilities of transitions

            p_LL = p_L * ρ

            p_LH = p_L * (1 - ρ)

            p_HL = p_H * (1 - ρ)

            p_HH = p_H * ρ

        # E[Δlog y] and E[(Δlog y)²]

            mean_Δ = p_LH * Δ + p_HL * (-Δ)

            mean_Δ_sq = p_LH * Δ^2 + p_HL * Δ^2

            var_Δ = mean_Δ_sq - mean_Δ^2

        println("ρ = $ρ \t\t $(round(var_Δ, digits=5))")
    end


# ==============================================================================
# Question 3
# ==============================================================================

# Set seed

    Random.seed!(1776)

# Generate random numbers for 1000 households and 8 years

    rand_mat = rand(1000, 8)

# Set parameters

    N = 1000
    T_sim = 8
    ρ_sim = 0.95


# Transition matrix for ρ = 0.95

    P_sim = transition_matrix(ρ_sim)
```

```julia
# Allocate state matrix: 1 = low income, 2 = high income
# Use column 1 of rand_mat to assign initial condition:
# ~20% start in low state, ~80% start in high state
    states = fill(2, N, T_sim)

    states[rand_mat[:, 1] .< 0.2, 1] .= 1

# Simulate forward using columns 2:T_sim of rand_mat
# For each household, compare rand draw to cumulative probability of staying in current state
# If rand < P[s,1] → go to state 1 (low), otherwise → state 2 (high)

    for t in 2:T_sim
        for i in 1:N

            # Get current state
            s = states[i, t-1]

            # Update state based on random draw and transition probabilities
            states[i, t] = rand_mat[i, t] < P_sim[s, 1] ? 1 : 2
        end
    end

# Compute share of high types in simulation at each period

    share_high_sim = vec(mean(states .== 2, dims=1))

# Compute share of high types from the continuum (theoretical) exercise
# Reuse the evolve_distribution function from Q1 with ρ = 0.95

    P95 = transition_matrix(0.95)
    π_cont = [0.2, 0.8]  # initial distribution [P(low), P(high)]
    shares_low_cont = evolve_distribution(π_cont, P95, T_sim - 1)
    share_high_cont = 1.0 .- shares_low_cont  # convert to high-type share

# Plot comparison

    plt3 = plot(1:T_sim, share_high_sim,

        marker=:circle, linewidth=2, label="1000 households",
        xlabel="Year", ylabel="Share of high-income households",
        title="Simulated vs. Continuum: Share of High Types (ρ=0.95)",
        size=(800, 500))

    plot!(plt3, 1:T_sim, share_high_cont,
        marker=:circle, linestyle=:dash, linewidth=2, label="Continuum")
```

```julia
# ==============================================================================
# Question 4
# ==============================================================================

# Convert states to income levels (1 = yL = 1, 2 = yH = 2)

    Y_panel = float.(states)
    Y_panel[states .== 1] .= yL
    Y_panel[states .== 2] .= yH

# Dependent variable y_{i,t} for t=2,...,8 and lag y_{i,t-1} for t=1,...,7

    Y_curr = Y_panel[:, 2:end]
    Y_lag  = Y_panel[:, 1:end-1]

# Stack into vectors

    y_vec     = vec(Y_curr)
    y_lag_vec = vec(Y_lag)

# --- Model 1: Pooled OLS ---

    X_ols = hcat(ones(length(y_lag_vec)), y_lag_vec)
     _ols = X_ols \ y_vec

    println("OLS:    = $(round( _ols[1], digits=4)),    = $(round( _ols[2], digits=4))")

# --- Model 2: Fixed Effects (within transformation) ---

    y_curr_mean = mean(Y_curr, dims=2)
    y_lag_mean  = mean(Y_lag, dims=2)

    Y_curr_dm = Y_curr .- y_curr_mean
    Y_lag_dm  = Y_lag  .- y_lag_mean

    y_dm     = vec(Y_curr_dm)
    y_lag_dm = vec(Y_lag_dm)

     _fe = dot(y_lag_dm, y_dm) / dot(y_lag_dm, y_lag_dm)

    println("FE:     = $(round( _fe, digits=4))")
    println("True   = 2ρ - 1 = $(2*0.95 - 1)")

# ===== Monte Carlo simulation: sampling distribution of ^ across ρ values =====

    function monte_carlo_beta(ρ; N=1000, T=8, reps=5000)
```

```
    _ols_vec = zeros(reps)
    _fe_vec  = zeros(reps)

    P_mc = transition_matrix(ρ)

    for r in 1:reps
        # Draw random matrix
        rm = rand(N, T)

        # Assign initial states using column 1
        s = fill(2, N, T)
        s[rm[:, 1] .< 0.2, 1] .= 1

        # Simulate forward
        for t in 2:T
            for i in 1:N
                s[i, t] = rm[i, t] < P_mc[s[i, t-1], 1] ? 1 : 2
            end
        end

        # Convert to income
        y = float.(s)
        y[s .== 1] .= yL
        y[s .== 2] .= yH

        # Current and lag
        Yc = y[:, 2:end]
        Yl = y[:, 1:end-1]

        # OLS
        yc_v = vec(Yc)
        yl_v = vec(Yl)
        X = hcat(ones(length(yl_v)), yl_v)
        _ols_vec[r] = (X \ yc_v)[2]

        # FE (demeaning)
        Yc_dm = Yc .- mean(Yc, dims=2)
        Yl_dm = Yl .- mean(Yl, dims=2)
        yc_dm = vec(Yc_dm)
        yl_dm = vec(Yl_dm)
        denom = dot(yl_dm, yl_dm)
        _fe_vec[r] = denom  0.0 ? NaN : dot(yl_dm, yc_dm) / denom
    end

    return _ols_vec, _fe_vec
end
```

```julia
# ===== Plotting parameters =====

# Run Monte Carlo for each ρ and plot histograms
for ρ in ρ_vals
    _ols_mc, _fe_mc = monte_carlo_beta(ρ)
    _fe_mc = _fe_mc[isfinite.(_fe_mc)]
    _true = 2ρ - 1

    # Combine all finite estimates to set axis range
    all_ = vcat(_ols_mc, _fe_mc)
    bmin, bmax = extrema(all_)
    margin = max((bmax - bmin) * 0.1, 0.01)
    xl = (bmin - margin, bmax + margin)
    edges = range(xl[1], xl[2], length=60)

    plt_mc = histogram(_ols_mc, bins=edges, alpha=0.6, normalize=true, label="OLS",
        xlabel="β̂", ylabel="Density",
        title="Sampling distribution of β̂ (ρ = $ρ)",
        size=(700, 450), xlims=xl)

    if length(_fe_mc) > 0
        histogram!(plt_mc, _fe_mc, bins=edges, alpha=0.6, normalize=true, label="FE")
    end

    vline!(plt_mc, [_true], color=:red, linestyle=:dash, linewidth=2,
        label="True β = $(round(_true, digits=2))")

    ρ_str = replace(string(ρ), "." => "_")
    savefig(plt_mc, joinpath(RESULTS_DIR, "fig_ex1_q4_ρ_$(ρ_str).pdf"))
    display(plt_mc)
end
```

Part 1 Exercise 2, Julia code

- Exercise 2 Main

```julia
# Exam MT 2025 Assignment 1
# Part 1 Exercise 2

# === Set up ===

    # Load packages

        using LinearAlgebra
        using Statistics
        using Plots
        using Printf

    # Set up paths

        const RESULTS_DIR = joinpath(@__DIR__, "..", "results")

    # Load code files for functions

        include("ex2_parameters.jl")
        include("ex2_vfi.jl")
        include("ex2_kfe.jl")

    # Set up parameters and ρ values

        p = Params()
        ρ_vals = [0.0, 0.05, 0.2, 0.5, 0.7, 0.95, 1.0]

# ============================================================
# Question 1
# ============================================================

asset_supply = Dict{Float64, Float64}()
solutions = Dict{Float64, Any}()

for ρ in ρ_vals
    println("\n=== Solving for ρ = $ρ ===")
    sol = solve_household(p, ρ)
    solutions[ρ] = sol

    if ρ == 1.0
        # ρ=1: absorbing states, no unique stationary dist
        # Under 50/50 split, both types hit borrowing constraint → B = -0.5
        asset_supply[ρ] = -0.5
```

```
        println("ρ=1.0: absorbing states → B = -0.5 (by argument)")
    else
         = stationary_dist(sol, ρ, p)
        B = agg_asset_supply( , sol)
        asset_supply[ρ] = B
        println("Asset supply: B = $B")
    end
end

# Print table
println("\n===== Q1: Asset Supply by ρ =====")
println("ρ \t\t Asset Supply")
println("-"^35)
for ρ in ρ_vals
    @printf("%.2f \t\t %.6f\n", ρ, asset_supply[ρ])
end



# ============================================================
# Question 2a: Counterfactual asset supply
# Keep c = c^{0.5} but use income process for ρ=0 and ρ=0.95
# ============================================================

# Benchmark solution: ρ = 0.5
sol_benchmark = solutions[0.5]

# Counterfactual: use benchmark policy but different ρ for income transitions
B_cf = Dict{Float64, Float64}()
for ρ_cf in [0.0, 0.95]
    # Use the benchmark solution's policy_idx, but compute stationary dist under ρ_cf
     _cf = stationary_dist(sol_benchmark, ρ_cf, p)
    B_cf[ρ_cf] = agg_asset_supply( _cf, sol_benchmark)
    println("B(ρ=$ρ_cf, c=c^0.5) = $(B_cf[ρ_cf])")
end



# ============================================================
# Question 2b: Decomposition table
# ============================================================

B_benchmark = asset_supply[0.5]

# ρ = 0 decomposition
T_total_0 = asset_supply[0.0] - B_benchmark
T1_0 = asset_supply[0.0] - B_cf[0.0]
```

```
T2_0 = B_cf[0.0] - B_benchmark

# ρ = 0.95 decomposition
T_total_95 = asset_supply[0.95] - B_benchmark
T1_95 = asset_supply[0.95] - B_cf[0.95]
T2_95 = B_cf[0.95] - B_benchmark

# Print decomposition table
println("\n===== Q2b: Decomposition =====")
println("ρ \t T_total \t T1 (%) \t\t T2 (%)")
println("-"^55)
@printf("0.00 \t %.4f \t\t %.1f \t\t %.1f\n", T_total_0, 100*T1_0/T_total_0, 100*T2_0/T_total_
@printf("0.95 \t %.4f \t\t %.1f \t\t %.1f\n", T_total_95, 100*T1_95/T_total_95, 100*T2_95/T_to
```

- Exercise 2 Parameters

```
# Part 1, Exercise 2: Parameters

# ex2_parameters.jl
using Parameters

Params = @with_kw (
    nb = 50,
    b_min = -0.5,
    b_max = 5.0,
    yL = 1.0,
    yH = 2.0,
      = 0.85,
      = 1.5,
    r = 0.10,
    tol_vfi = 1e-6,
    tol_kfe = 1e-6,
    max_iter = 500
)

trans_matrix(ρ) = [ρ (1-ρ); (1-ρ) ρ]
```

- Exercise 2 Value Function Iteration

```
# Part 1, Exercise 2: Value Function Iteration

# ex2_vfi.jl

function solve_household(p, ρ::Float64)

    # Unpack

        nb = p.nb
        b_grid = collect(LinRange(p.b_min, p.b_max, nb))
        y_grid = [p.yL, p.yH]
        ny = 2
        Π = trans_matrix(ρ)

    # CRRA utility

        function u(c)
            c > 0 ? (c^(1 - p. )) / (1 - p. ) : -1e10
        end

    # Pre-compute consumption for all (b, y, b') triples
    # c(b, y, b') = (1+r)*b + y - b'
    # Store as matrix: rows = (b,y) states, cols = b' choices

        n_states = nb * ny
        c_mat = zeros(n_states, nb)
        for iy in 1:ny
            for ib in 1:nb
                idx = (iy - 1) * nb + ib
                for ib_next in 1:nb
                    c_mat[idx, ib_next] = (1 + p.r) * b_grid[ib] + y_grid[iy] - b_grid[ib_next]
                end
            end
        end

    # Utility matrix

        u_mat = u.(c_mat)

    # Initialise value function

        V = -ones(n_states)
        V_new = similar(V)

        # stores index of optimal b'
```

```
    policy_idx = zeros(Int, n_states)

# VFI loop

converged = false
for iter in 1:p.max_iter
    # Compute expected continuation value: EV(b') = Σ_y' Π(y,y') V(b',y')
    # V is ordered as [V(b1,y1), V(b2,y1),...,V(bN,y1), V(b1,y2),...,V(bN,y2)]

    # nb × ny
        V_mat = reshape(V, nb, ny)

    # nb × ny (expected V for each b' and current y)
        EV_mat = V_mat * Π'

    # back to vector, same ordering as V
        EV = vec(EV_mat)

    # Maximise over b' for each state (b, y)

    for iy in 1:ny

        # EV(b', y) for current y
            EV_slice = EV[(iy-1)*nb+1 : iy*nb]
            for ib in 1:nb
                idx = (iy - 1) * nb + ib
                best_val = -Inf
                best_ib = 1
                for ib_next in 1:nb
                    val = u_mat[idx, ib_next] + p.  * EV_slice[ib_next]
                    if val > best_val
                        best_val = val
                        best_ib = ib_next
                    end
                end
                V_new[idx] = best_val
                policy_idx[idx] = best_ib
            end
    end

# Check convergence

    if maximum(abs.(V_new - V)) < p.tol_vfi
        V .= V_new
        converged = true
        println("VFI converged after $iter iterations (ρ=$ρ)")
        break
```

```
        end
        V .= V_new
    end

    if !converged
        println("VFI did NOT converge for ρ=$ρ")
    end

    # Extract policy functions
        b_next = [b_grid[policy_idx[i]] for i in 1:n_states]
        c_policy = [(1 + p.r) * b_grid[mod1(i, nb)] + y_grid[div(i-1, nb)+1] - b_next[i] for i

    return (V=V, policy_idx=policy_idx, b_next=b_next, c_policy=c_policy,
            b_grid=b_grid, y_grid=y_grid, nb=nb, ny=ny, converged=converged)
end
```

- Exercise 2 Kolmogorov-Forward Equation

```
# Part 1, Exercise 2: Kolmogorov-Forward Equation
# kfe.jl - Compute stationary distribution over (b, y) states

function stationary_dist(sol, ρ::Float64, p)
    nb = sol.nb
    ny = sol.ny
    n_states = nb * ny
    Π = trans_matrix(ρ)

    # Build full transition matrix over (b, y) → (b', y')
    # For state i = (b, y): policy says go to b' = b_grid[policy_idx[i]]
    # Then y transitions according to Π
    T = zeros(n_states, n_states)

    for iy in 1:ny
        for ib in 1:nb
            idx_from = (iy - 1) * nb + ib
            ib_next = sol.policy_idx[idx_from]
            for iy_next in 1:ny
                idx_to = (iy_next - 1) * nb + ib_next
                T[idx_from, idx_to] = Π[iy, iy_next]
            end
        end
    end

    # Iterate to find stationary distribution:  ' = T' *
     = ones(n_states) ./ n_states  # uniform initial guess

    for k in 1:p.max_iter
         _old = copy( )
          = T' *
        if maximum(abs.( - _old)) < p.tol_kfe
            println("KFE converged after $k iterations")
            break
        end
    end

    return
end

# Compute aggregate asset supply: B = Σ (b,y) * b
function agg_asset_supply( , sol)
    b_grid_large = vcat(sol.b_grid, sol.b_grid)  # repeat for each y
    return dot( , b_grid_large)
end
```